

EV Challenge — GoodWe

Chatbot com IA para mobilidade elétrica — orquestrando o futuro com dados, IoT e IA.

CHALLENGE INTEGRADOR · SPRINT 03

DISCIPLINA	Prompt and Artificial Intelligence
CURSO	Ciência da Computação — 2º Semestre (2SEM)
PARCEIRO	GoodWe Brasil
PERÍODO	Semestre 2026.2
PROFESSOR	Jorge Luiz Gomes
FORMATO	Trabalho em grupo · Sprint 03 · 100 pts + bônus
ENTREGA	21/09/2026 · repositório público + relatório de evolução (PDF)
CONTINUIDADE	Evolução do projeto das Sprints 1 e 2 (semestre anterior)

1. Contexto e premissa

O grupo **já entregou** a **Sprint 1** (exploração e planejamento) e a **Sprint 2** (desenvolvimento e entrega) do chatbot EV Challenge — GoodWe. A **Sprint 03 continua o mesmo projeto** — agora com a stack do **Módulo 1** (Aulas 01–04): LangChain LCEL, memória conversacional, structured output Pydantic v2 e context engineering.

 **Projeto local**

Os projetos dos alunos são **locais, não notebooks**. A evolução acontece **dentro do projeto existente**, criando-se apenas as pastas indicadas neste documento — o grupo é livre para evoluir o código como preferir.

 **Relação com os CKPs**

Os checkpoints **preparam** o grupo para o Challenge (CKP01 = trampolim do LangChain, 24/08; CKP02 = do RAG, 21/09). O Challenge é o **projeto obrigatório em grupo** que vai além do checkpoint. Sprints e CKPs convivem — a entrega da Sprint 03 (21/09) é distinta do CKP02 entregue na mesma data.

2. Conceito — Refactory conversacional em LangChain

Reconstruir o **núcleo conversacional** do chatbot em **LangChain LCEL**: chain `prompt | llm | parser`, **memória por sessão** com limite de tokens, **saída estruturada Pydantic v2** validada e **context engineering** leve (XML tagging). O ganho do refactory é demonstrado por um **comparativo antes/depois**: versão manual das Sprints 1/2 vs. versão refatorada em LangChain.

3. Escopo — o que é exigido (só o que a turma viu no M1)

#	ITEM	DETALHE	ORIGEM
1	Chain LCEL end-to-end	<code>ChatPromptTemplate ChatOllama (gpt-oss:120b) parser</code> reconstruindo o núcleo do chatbot	Aula 01
2	Memória por sessão	<code>RunnableWithMessageHistory</code> + <code>ConversationTokenBufferMemory</code> , limite de tokens, demonstrada em 3+ turnos	Aula 02
3	Structured output	schema Pydantic v2 do domínio EV com <code>field_validator</code> (ex.: <code>ConsultaRecarga</code>)	Aula 03
4	Context engineering	system prompt versionado (XML tagging) + medição de tokens com <code>tiktoken</code>	Aula 04
5	Segurança e guardrails	recusa de jailbreak/prompt injection; validação de escopo GoodWe (ver §6)	Transversal
6	Relatório de evolução do projeto	até 5 páginas, com tabela de comparativo antes/depois (ver §8)	—

NÃO EXIGIR LangGraph, agentes, function calling novo, RAG, interface web e observabilidade — pertencem aos Módulos 3 e 4.

BÔNUS +1 PT chamada multi-provider (consultar mais de um modelo e mais de um prompt).

4. Mapeamento Módulo 1 → Sprint 03

AULA	CONTEÚDO	USO NA SPRINT
01 · 03/08	Revisão LangChain: LCEL, ChatOllama, Output Parsers	chain base <code>prompt</code> <code>llm</code> <code>parser</code>
02 · 10/08	Memória Conversacional	memória por sessão com limite de tokens e política de resumo
03 · 17/08	Structured Output com Pydantic v2	saída estruturada validada (schema do domínio, <code>field_validator</code>)
04 · 24/08	Context Engineering CKP01	prompt v2 com XML tagging + medição de tokens (<code>tiktoken</code>)

5. Pastas a criar no projeto local existente

- `prompts/` → system prompt **versionado** (`system_prompt_v1.md` ... `vN`) + tabela de versões (o que mudou, por quê, ganho medido)
- `src/chain/` → `builder.py` (LCEL) e `memoria.py`
- `src/schemas/` → schema Pydantic v2 do domínio EV (ex.: `ConsultaRecarga` — estado do carregador, potência, faturamento)
- `src/guardrails/` → `scope_validator.py` e `moderation.py`
- `evals/` → eval set reexecutado (happy path, edge cases, jailbreak, out-of-scope) + `sprint3_results.json`
- `docs/` → **relatório de evolução do projeto** (PDF, até 5 pág.) + `relatorio_modelos.md`

6. Pontos obrigatórios

- System prompt versionado** — tabela de versões com o que mudou, por quê e o **ganho medido**.
- Relatório de uso de modelos e parâmetros** (`relatorio_modelos.md`) — comparar **2+ modelos** (ex.: `gpt-oss:120b` vs. `qwen3:8b`) e documentar `temperature`, `top_p` e `max_tokens`.
- Segurança e guardrails do sistema** — escopo GoodWe: **não inventar especificações de produtos** não inseridas na base; recusa de aconselhamento jurídico, financeiro ou de segurança elétrica sem orientar profissional habilitado; recusa de jailbreak/prompt injection.
- Relatório de evolução do projeto (até 5 páginas)** — estrutura em §8.
- Bônus (+1 pt)** — chamada **multi-provider**: consultar mais de um modelo e mais de um prompt.

7. Rubrica — Sprint 03 (100 pts + bônus)

BLOCO	PONTOS	CRITÉRIOS
A — Refactory LangChain	40	chain LCEL end-to-end; memória funcionando em 3+ turnos; structured output validado pelo schema Pydantic v2
B — Prompt versionado + relatório de modelos	25	tabela de versões com ganho medido; comparativo de 2+ modelos com parâmetros documentados
C — Segurança e guardrails	15	jailbreak/injection recusados; escopo GoodWe validado; recusas de domínio respeitando orientação a profissional habilitado
D — Eval, evolução e relatório	20	eval reexecutado; relatório de evolução (até 5 pág.) com tabela antes/depois ; iterações documentadas
Bônus — multi-provider	+1	chamada com mais de um modelo e mais de um prompt

8. Relatório de evolução do projeto (obrigatório · até 5 páginas)

Entregue em PDF junto do repositório, na pasta `docs/`. Estrutura mínima:

1. **Resumo da evolução** — o que existia nas Sprints 1/2 → o que existe agora na Sprint 03.
2. **Refatoração** — decisões técnicas e trade-offs da migração do núcleo conversacional para LCEL.
3. **Tabela de comparativo antes/depois (OBRIGATÓRIA)** — colunas "Sprints 1/2 (versão manual/legado)" × "Sprint 03 (LCEL)", com no mínimo: qualidade das respostas (nota no eval), tokens por turno, latência média e acurácia do structured output.
4. **Problemas encontrados e soluções** — pelo menos 2, com a decisão tomada e o porquê.
5. **Equipe e divisão de trabalho** — quem fez o quê (nome, RM, tarefa principal).

🎯 Como provar o "ganho"

O mesmo eval set das Sprints 1/2 deve ser reexecutado sobre a versão refatorada. A tabela antes/depois é a evidência central do refactory — sem ela, o bloco D da rubrica perde a maior parte dos pontos.

9. Cronograma

DATA	EVENTO
03/08 – 24/08	Módulo 1 (Aulas 01–04) · CKP01 em 24/08 (individual)
24/08 → 21/09	~4 semanas de desenvolvimento do refactory em LangChain
21/09	ENTREGA SPRINT 03 · repositório atualizado + relatório de evolução (PDF)

⚠️ Atenção à data

A entrega da Sprint 03 cai **na mesma data do CKP02** (21/09, Aula 07). São entregas **distintas**: o CKP02 é o entregável do checkpoint; a Sprint 03 é o projeto em grupo do Challenge.

10. Condições de entrega

- Repositório **público** com acesso ao professor; histórico Git com commits regulares de cada integrante.
- **Sem API key no histórico** — credenciais via variável de ambiente, `.env` (gitignored) ou Colab Secrets. Exposição implica penalidade.
- Entrega por **link do repositório + arquivo .txt** com nome, RM e turma de cada integrante.
- O **relatório de evolução do projeto** (até 5 páginas) é parte obrigatória da entrega desta sprint.

11. Integridade acadêmica

- Uso de IA generativa é incentivado, mas **não** substitui a entrega: o projeto é do grupo.
- Submeter o repositório de outro grupo ou de semestres anteriores = **zero** na sprint e processo disciplinar.
- Auto-plágio entre disciplinas não é permitido sem autorização.

Documento oficial do Challenge — Sprint 03 · versão 1.0 · leitura obrigatória

Classificação: Uso Acadêmico FIAP · Prof. Jorge Luiz Gomes · FIAP · Ciência da Computação · 2026.2